

ACCOUNTS_DESIGN_PROPOSAL

Helix OTA Multi-Tenant Accounts — Design + Phased Implementation-Plan Proposal

Revision: 1 **Last modified:** 2026-07-12T00:00:00Z **Status:** PROPOSAL — design-first, brainstorming HARD-GATE. NO implementation until the operator approves this plan AND resolves the §7 decision gate (§11.4.167 isolated big-feature stream; §11.4.6/§11.4.66 no-guess/surface-don't-decide). **Authority:** operator directive 2026-07-10 (multi-account mandate) · docs/planning/PRODUCTION_READINESS_PLAN.md §2.1/§5 P3/K12 · the 11-doc research set docs/research/accounts/00_INDEX.md ... 30_delivery_plan.md **Scope owner / track:** T2 / feature/multi-tenant-accounts (§11.4.181 canonical branch; supersedes the config's feature/accounts-web label — reconcile per PRODUCTION_READINESS_PLAN §2.9)

Relationship to the research set (§11.4.186 — no divergent second source of truth). This is the CONSOLIDATED, actionable design + plan the operator reviews at the approval gate. It **synthesizes and sequences** the detailed research/design set; it does **not** redefine any entity, claim, route, or test shape. The authoritative single-sources-of-truth remain:

20_target_multitenancy_data_model.md (entities/columns/migration/RLS), 21_authz_rbac_superadmin.md (permission model + token claim, OQ-2), 22_api_surface.md (routes), 26_security_threat_model.md (threats + the isolation proof + 404-vs-403), 27_test_strategy.md (tests), 30_delivery_plan.md (milestones), 28_reconciliation_and_open_decisions.md (the §11.4.186 cross-doc pass + decision register). Where an SSOT doc finalizes a different shape, **that doc wins** and this proposal is reconciled to it. Every “as-is” fact below carries a file:line re-

read this session; facts inherited from the research set (not re-opened here) are marked **[inherited]**. This is a §11.4 planning artifact — nothing here is a completion claim.

0. Executive summary + the load-bearing isolation invariant

Helix OTA today is a **single-tenant** control plane: one hosted instance serves one logical fleet. The mandate is to serve **many accounts (organizations) from one hosted instance**, super-admin-provisioned, with per-account data/RBAC/key isolation and account-selection after sign-in. The research set establishes (and this session re-verified) that this is primarily a **data-model + scoping** effort, not a from-scratch auth build — authN, RBAC, a super_admin-aware SPA, and a per-project ACL already exist; the missing piece is a tenant layer **above** project and the scoping of every OTA resource to it.

THE load-bearing security invariant this whole feature exists to guarantee:

A principal authenticated to account **A** can **NEVER** read or mutate account **B**'s resources (devices / artifacts / releases / deployments / telemetry / groups / audit), and a device enrolled to account **A** is **never** offered account **B**'s deployment — even when their (os, model, group) match. This is enforced by **three independent layers** (compile-time explicit accountID params → app-layer requireAccountAccess middleware → PostgreSQL RLS), and **proven** by a paired §1.1 mutation that removes each layer's scoping and makes the isolation test **FAIL**, run against **BOTH** store backends. Isolation asserted at only one layer is not proven. (26_* §1, 27_* §2.)

Everything below serves that invariant.

1. Tenancy model

1.1 What “account / tenant” means here

An **account** is the top-level **tenant** — an organization/customer of the hosted platform. It has no parent. It sits **above** the existing Project:

Account (tenant, NEW)	accounts
└ Project (gains account_id FK)	projects.account_id →
accounts	
└ OTA updates:	each resource carries
account_id (RLS key) + project_id	
Artifact · Release ·	
Deployment · Device ·	
Group · Telemetry · Audit	

Users are a **single global identity** that may belong to **one or more accounts** (mandate 00_INDEX §1.3); the per-account role lives on a **membership** join, not on the user row, so the same person is admin in account A and viewer in account B. A **super-admin** is a global identity property (is_super_admin), not a per-account role, that “sees/controls everything”. Account creation is **super-admin-only** — there is no self-registration wizard (00_INDEX §1.2).

1.2 The isolation boundary that MUST hold — and the storage model that enforces it

Chosen tenancy storage model (SSOT 20_* §0): **shared PostgreSQL schema + a denormalized account_id column on every tenant-owned row, defended by Row-Level Security (RLS)** — the industry default (Microsoft Azure Architecture Center / AWS / WorkOS, verified in 20_*) when tenant counts grow and no per-tenant schema customization is required. It also demands the **least** change to the shipped single-schema store. Database-per-tenant / schema-per-tenant are reserved as a future escalation for a single high-compliance/residency account, forward-compatible via the account_id seam.

The isolation boundary is **defense-in-depth across three layers** (a bug in one is caught by another — AWS: RLS “works even when your application code has bugs”):

Layer	Mechanism	What it guarantees
L2 compile-time	explicit accountID param on every scoped Repository get/create/update + AccountID/ProjectID on the *Filter structs (20_* §3.2)	a forgotten scope is a build error , not a runtime leak
L1 app middleware	requireAccountAccess(minRole) runs after authMiddleware; reads account_id from	a request whose

Layer	Mechanism	What it guarantees
L3 database RLS	the verified token claim , re-verifies membership, denies below role (21_* §2.3, 22_* §2.1)	token account ≠ target, or role < minimum, or account suspended, is rejected
	pgx sets app.current_account GUC per request; CREATE POLICY ... USING (account_id = current_setting('app.current_account')) (20_* §4/§6)	the DB refuses cross-account rows even if L1/L2 were bypassed by a bug

Critical caveat (danger zone, 30_* §3.2 / 27_* §0.1): RLS (L3) exists ONLY on the pgx path; the shipped default store is **in-memory** (config.go:112-116 DatabaseURL selects pgx, unset ⇒ memory — verified). On the memory MVP backend, **L1+L2 are the entire isolation guarantee** — which is exactly why the L2 explicit-accountID param (compile-time) is chosen over a context-implicit scope. The RLS test layer is **pgx-only** and honestly **SKIP-with-reason: topology_unsupported** when no live Postgres is booted (never PASS-by-default) — boot a real Postgres on-demand via the containers submodule (rootless podman §11.4.161).

1.3 The trust boundary the account claim MUST inherit (single most important rule)

The codebase already refuses to derive trust from a request: the artifact verify key comes ONLY from server config, never a request field (resolvePublicKey, handlers_artifact.go — “*There is deliberately no request path into this function*” [inherited, 11_*/26_*]); TrustTLSProxy is an explicit operator boolean, never inferred from X-Forwarded-Proto (config.go:88-110, verified). **The account-tenancy claim MUST follow**

the identical rule: server-minted, server-verified, NEVER self-asserted by the caller. This is the single load-bearing security principle of the whole feature (26_* §0).

1.4 Current state (verified this session — where the account layer attaches)

- **No account/org/tenant entity anywhere.** The `store.Repository` interface (`store.go:315-429`) has a `Projects` block but every OTA accessor (`Device/Artifact/Release/Deployment/Telemetry/Group/Audit`) is **unscoped**: `LatestRelease(os, targetModel)` (`store.go:349`), `ActiveDeploymentForTarget(os, targetModel, group)` (`:359`), `GetDeviceByHardwareID(hardwareID)` (`:338`) — none takes an `account/project`. `DeviceFilter/ReleaseFilter/AuditFilter` (`store.go:257-285`) carry no `AccountID/ProjectID`.
 - **Projects are empty containers.** `Project/ProjectRole/ProjectAccess` exist (`store.go:287-311`) and the `Project` doc-comment even claims “*providing multi-tenant isolation*” (`:288`) — but that is **aspirational**: no OTA resource carries a `ProjectID`, so the `project → resource` link is entirely absent. The `ProjectAccess ACL + GetProjectAccess/ SetProjectAccess/ListProjectMembers/ RemoveProjectAccess` (`store.go:322-332`) is the **exact template** the account layer lifts one level up.
 - **No persisted user.** Identity is the bare token `Subject`; the directory is the in-memory `StaticUserDirectory` (`users.go:8-48`, constant-time compare `:37-47`).
 - **Token carries no tenant dimension.** `Claims{Subject, Roles, IssuedAt, Expiry}` (`token.go:31-36`) — no `account_id`. Roles `admin|operator|viewer|device` (`token.go:14-20`) — **no super_admin**.
 - **Every OTA route is gated only by the GLOBAL role** (`requireRole, middleware.go:82-97`) — no `account/project` data-level gate. `requireProjectAccess` exists (`handlers_project.go:37-74`, `global-admin bypass :43-60`, `viewer<operator<admin` rank compare `:78-92`) but is wired onto the `/projects` routes only.
-

2. Data model changes

SSOT: 20_* (entities, DDL, migration). Summary of what lands; full DDL is 20_* §4.

2.1 New / lifted entities (four)

Entity	Purpose	Key fields (20_* §1)
Account (tenant, top of tree)	the organization boundary	account_id PK · name UNIQUE · slug UNIQUE (stable switch key, §11.4.111) · status ∈ {active, suspended, archived} · timestamps
User (persisted identity — closes the no-persisted-user gap)	one global identity, many accounts	user_id PK · username UNIQUE · email UNIQUE · password_hash · is_super_admin (global bypass flag, bootstrap-only) · is_active
AccountMembership (user ↔ account M:N)	per-account role	composite PK (user_id, account_id) · role ∈ {viewer, operator, admin} · is_owner · granted_at/ granted_by
Role/Permission (per-account authz vocabulary)	RBAC skeleton now; reserved tables for “max flexibility” later	ships fixed Membership.role enum now; roles/ permissions/ role_permissions tables as commented DDL — turned on additively only if OQ-2 chooses table-based RBAC (no 2nd data-model change)

Id type: opaque **TEXT**, app-minted — mirrors the shipped store (project_id/device_id/artifact_id are all **TEXT**), NOT the UUID of the canonical 001-style design-target schema. 20_* §1/§4 states the **TEXT** ↔ **UUID** mapping so the two representations never silently diverge. (Secondary decision, §7.)

2.2 Scope every OTA table

Add a denormalized **account_id** (NOT NULL after backfill — the **RLS key + leading index column, AWS rule**) and a **project_id** to Device, Artifact, Release, Deployment, Telemetry, Group, Audit (nullability per 20_* §2.1). Enforce resource.account_id == project.account_id with a **composite FK** (project_id, account_id) → projects(project_id, account_id) so the DB — not application discipline — guarantees the two never drift (20_* §2, original work). Re-scope the 7 **single-global-**

tenant assumptions (20_* §2.2): project name →
UNIQUE(account_id,name); device hardware_id →
UNIQUE(account_id,hardware_id); release monotonicity →
(account_id,project_id,os,model); active-deployment uniqueness →
(account_id,project_id,os,model,group); group name →
UNIQUE(account_id,name); ProjectAccess.CallerID → user_id; token
gains a tenant claim.

2.3 store.Repository extension — memory ↔ pgx parity is MANDATORY

New interface methods mirror the Projects block and are implemented by **BOTH** MemoryRepository and PostgresRepository: Accounts CRUD, the User CRUD that does not exist today, Memberships, and an **atomic CreateAccountWithOwner(ctx, account, ownerUserID, role)** that closes the orphan-tenant hole (a crash between “account created” and “owner membership granted” would otherwise leave an un-administerable tenant — 20_* §3.3, recommend the purpose-built atomic method now, generic WithTx later).

Parity is a first-class requirement, not an afterthought (this session’s HB-2/STORE-1 lesson). The memory backend has no RLS, so its isolation rests entirely on L1+L2 — a memory-vs-pgx behavioral divergence is a silent isolation gap. The project already carries a **shared contract test suite** (store/contract_test.go, store/contract_fabric_test.go) that runs the identical assertions against both backends; HB-2 (03340896) closed exactly such a memory-vs-Postgres fabric-lease divergence by making the behavior **uniform**. Every new account-scoping method **MUST** be added to that shared contract suite so both backends are proven byte-identical in behavior, and the flagship isolation test (§5) runs against both.

2.4 Migration approach (OQ-4 — operator decision, §7)

Recommended: default-account backfill (big-bang, then NOT NULL) — 20_* §5 Option A: 002 adds account_id nullable → one-shot backfill of every legacy row to a seeded __default__ account → 003 sets NOT NULL, adds per-account uniques, the composite FKs, leading-column indexes, and enables RLS. It removes the nullable-isolation-hole window entirely, and is **near-free** here because the store is in-memory-by-default and barely populated. Every destructive 003 step carries a §9.2 hardlinked backup + defined expected post-op state + restore-on-fail

gate. **Flip to Option B** (additive-nullable-until-backfill, NULL-rows super-admin-only) ONLY if a target already holds meaningful prod data AND zero-downtime is mandatory.

2.5 [Resolved this proposal] `api_keys.project_id` — reconciliation #H

28_* §1.3 flags a **RESIDUAL-BLOCKING** gap: three docs (21_* §6.1, 22_* §4.1, 24_* §1.1) rely on an `api_keys.project_id` scope column (“NULL = any project in the account”) that the 20_* §4 DDL omits. **This proposal adopts the fix:** the M1 DDL adds `api_keys.project_id` TEXT nullable (REFERENCES projects(project_id)). Left unfixed, implementers would re-invent the column ad hoc — the divergence §11.4.186 forbids.

3. AuthN / AuthZ

SSOT: 21_* (permission model, token claim, OQ-2) + 26_* (threat proofs).

3.1 Permission model (OQ-2) — RBAC-first role+scope hybrid

Recommended (21_* §1.2): a role+scope hybrid, delivered RBAC-first, evaluated in this fixed order:

1. **Tenant-isolation predicate (always first).** The token’s active `account_id` MUST equal the resource’s `account_id`; only a super-admin bypasses. This is the L1 twin of L3’s RLS (AWS Verified Permissions “keep tenant isolation at the top”).
2. **RBAC role check.** The membership role (viewer<operator<admin) must meet the resource×action minimum — the **shipped client-side RESOURCE_PERMISSIONS grid promoted to server-authoritative** (today it is UX-only; 21_* §1.3), so client and server share one matrix.
3. **Thin ABAC deny-override.** A closed set of attributes only ever *subtracts* access: `account status='suspended', user is_active=false, key revoked_at IS NOT NULL`.

“Ultimate flexibility” grows additively via the reserved roles/permissions tables (no 2nd data-model change, no policy engine). Full ABAC (Cedar/OPA) stays a documented future escalation. Rejected: pure RBAC (role explosion) and pure ABAC now (a policy-engine dependency before a single tenant exists).

3.2 Token scoping — extend Claims with a server-minted account_id

Extend the shipped Claims{sub,roles,iat,exp} (token.go:31-36) with a **custom, top-level, immutable account_id claim** (device/CLI tokens also project_id), read only from the **verified** payload at the one gateway (authMiddleware), never from a header or body (21_* §3, multi-tenant-saas.com verified). Signing is unchanged — HMAC-SHA256, secret **from config only** (token.go, config.go:66-70); a larger payload, same MAC. The account claim is **derived on demand** via an account-selection exchange, never embedded in the long-lived refresh credential.

Belt-and-suspenders (reconciles 20_* “resolve-per-request” with 21_* “stamp-claim”): stamp the account_id claim at account-selection time (the fast hot-path hint) **AND** requireAccountAccess re-verifies GetAccountMembership(userID, account_id) on **every** request (the authority) — a stale/forged claim cannot outlive the membership row.

3.3 [Resolved this proposal] Legacy tokens — reconciliation #J (fail-closed)

28_* §1.3 flags the second **RESIDUAL-BLOCKING** item: 21_* §3.3 (fail-closed) and 22_* §6.1 (default-account token fallback) prescribe **opposite** security postures for a legacy no-account_id token. 22_* cedes authZ to 21_*. **This proposal adopts fail-closed (21_* §3.3 Option A):** the account_id claim is **optional in the struct** so legacy tokens still parse/verify, but a token with no account_id is **denied on every account-scoped route** (super-admin exempt); the 15-min access TTL ages legacy tokens out within one window. The 20_* §5 **data** backfill (existing rows → __default__) stays; the legacy-token → __default__ mapping is **struck** (it would silently widen scope — the exact NULL-scoping hole).

3.4 Super-admin — global identity flag, bootstrap-only, policy-based DB bypass

The super-admin is the global `users.is_super_admin` boolean (§1.1) — bypasses the L1 tenant predicate and the L3 RLS via a **policy-based bypass** (a super-admin GUC the `USING` clause also honors), **NOT a BYPASSRLS role** (AWS: policy-based keeps admin access *auditable and revocable*; a `BYPASSRLS` role is invisible to policies). Set **only via config/env bootstrap, never a request** — the same trust boundary as the `token secret / resolvePublicKey / TrustTLSProxy`. Every super-admin action writes an audit row naming the **affected account_id** + the real `user_id` (both empty today — `audit_wire.go` [inherited]). Treated as a **break-glass** identity (few, named, every use audited).

3.5 Enforcement stack + the tenant-isolation test that MUST pass

Middleware chain on every scoped route: `authMiddleware` (verify token, extract `account_id`) → **`requireAccountAccess(minRole)`** (NEW — re-verify membership, reject suspended/inactive, stash {`AccountID`, `ProjectID`}) → `requireProjectAccess` (existing pattern, now on all resource routes) → `requireRole`. A **`requireSuperAdmin`** middleware gates the `/admin/*` namespace.

The invariant a test MUST prove (§0): a principal scoped to account A issuing each cross-account vector (list, get-by-id, write, device-update, audit) against account B's ids receives deny/empty on **both** backends; a paired §1.1 mutation that drops the scope makes the test **FAIL**; and (pgx) a raw `SELECT` with a forged/absent `WHERE account_id` returns **zero** of B's rows (RLS holds when app code is wrong). 26_* §1.3/§2.4 owns this proof.

3.6 SEC-1 interplay — fail-fast on the token secret is a hard prerequisite

Verified this session: `config.go:180-184` **still silently falls back** to the literal `"helix-ota-dev-token-secret-change-me"` when `HELIX_TOKEN_SECRET` is unset. Under multi-account this is **catastrophic**: a predictable signing key lets an attacker mint a token carrying **any** `account_id` claim and defeat tenant isolation entirely (26_* §3.4/§6 Spoofing — the single highest-severity as-is finding). The **SEC-1 token-secret fail-fast** (refuse to boot on default/unset secret) is in-flight on **T1/main** (`PRODUCTION_READINESS_PLAN` §2.7 / P1 / item 12). **This**

proposal binds: the account claim (M2) **hard-depends** on SEC-1 landing first — pull fail-fast into M2 if it has not landed on T1 by then (30_* §3.2). This composes with, but is distinct from, K12's **per-account signing keys** (§7-5): SEC-1 protects the *token* signing secret; per-account keys protect the *artifact* verify keys (resolvePublicKey(accountID) registry, 26_* §4).

4. API surface

SSOT: 22_*. Additive-only under the existing /api/v1 group — **no /api/v2** (adding endpoints / optional fields / optional params are non-breaking, Speakeasy verified). Reuses the shipped `ErrorBody{code,message,details}` envelope + closed code set (`errors.go`) and the existing cursor+limit pagination — no new codes.

4.1 New / changed endpoints

Group	Endpoints	AuthZ
Super-admin admin API (new /admin/*)	POST/GET/GET/ PATCH/DELETE / admin/ accounts[:id] · .../users[:id] · PUT/GET/DELETE / admin/ accounts/:id/ members/:userId	requireSuperAdmin
Account selection (post-sign-in)	GET /auth/ accounts (list my memberships → picker) · POST / auth/select- account (exchange identity token → account-scoped token)	authMiddleware
Credentials	POST/GET/ DELETE /admin/ accounts/:id/ api-keys	mint: super-admin/account-admin; exchange: presents key

Group	Endpoints	AuthZ
	(cleartext once, key_hash stored) · POST /auth/ token-exchange (key → short-lived scoped bearer) · POST/ DELETE /admin/ accounts/:id/ signing-keys (per-account verify key, 26_* §4)	
Missing endpoint (load-bearing gap)	GET /artifacts (list) — absent today; the SPA release-picker + CLI block on it	requireAccountAccess(viewer)
Existing OTA routes — paths unchanged, gained scope	devices/artifacts/ releases/ deployments/ groups/audit/ client-update/ telemetry + / projects	requireAccountAccess(minRole); scope from the claim , never the path

4.2 Where tenancy lives in the request (the anchor decision)

Token-claim on the hot path (primary); path-tenancy for cross-account admin only (22_* §2.0). The existing OTA paths do **not** change — the scoping middleware reads `account_id` from the verified claim; only `/admin/accounts/:id/...` names a target account explicitly (authorized against `is_super_admin`). The project dimension stays the caller-supplied `--project`, **validated against the token's authorized set** — a narrowing within the claim's account, never a widening.

4.3 Backward-compat for single-tenant callers

Legacy single-tenant callers keep the same URLs; the account is derived server-side. Per §3.3 (reconciliation #J), a legacy **token** with no claim is **fail-closed on scoped routes** (not silently mapped to `__default__`); the **data** backfill maps legacy *rows* to `__default__`. Optional new response fields (`account_id/project_id`, a release notes pass-through) follow the §11.4.104(C)-style “legacy fixtures without the field still parse” rule. Cross-tenant denial returns **NOT_FOUND** (anti-enumeration) rather than 403 — recommended, hard rule owned by 26_* §6.2 (secondary decision, §7).

Out of scope of the control surface (flagged, not silently dropped): the artifact **byte download / object-storage** surface — bytes are validated then **discarded**, `StorageRef` is a placeholder [inherited, 11_*/26_* §5]. Real per-account object storage + signed URLs is an M7 dependency; until it lands, byte-level e2e is honestly `SKIP`: `object_storage_absent`.

5. Test strategy (anti-bluff §11.4.27 / §11.4.169 / §11.4.107)

SSOT: 27_*. Every applicable §11.4.169 test type × 6 surfaces (Server API · Data/RLS · CLI · device · dashboard SPA · OTA-Manager SPA); every PASS cites captured evidence under `docs/qa/<run-id>/` via `ab_pass_with_evidence` (§11.4.69), never a bare `ab_pass`; every gate ships a paired §1.1 mutation; deterministic at `-count=3`; real Postgres / real server / real device emulator / real MinIO — no fakes beyond unit (§11.4.27).

5.1 THE flagship — cross-account isolation, proven at three layers

The single most important proof (the §0 invariant). Each layer uses a **three-part not-a-false-negative assertion in the same call** (a broken query returns empty for *everyone*): (i) A’s own rows are **present + non-empty**, (ii) B’s rows are **absent**, (iii) the paired §1.1 mutation **flips (i)/(ii)** — B’s rows appear.

- **L3 DB / RLS** (pgx, live Postgres via rootless podman): session `SET app.current_account = A` → forged-WHERE `account_id = 'B'` read returns **0 rows**; cross-account `UPDATE/DELETE` affect **0 rows**.

Mutation: drop/weaken the policy → forged-WHERE read returns B's rows → **FAIL**. SKIP: topology_unsupported if no Postgres.

- **L1/L2 API** (real ota-server, httptest, **both backends**): A's token on GET /devices|/releases|/deployments|/artifacts returns only A; a known B id → deny (NOT_FOUND); cross-project/-account key on write → 403; account taken from the resolved scope, never a request field. Mutation: remove requireAccountAccess / drop the AccountID filter → A sees/mutates B → **FAIL. This is the memory MVP's only isolation layer** — it is where the default-backend safety is proven.
- **E2E** (real journey): user in A sees only A's data; device enrolled to A polling GET /client/update gets A's offer and 204/deny for B's deployment even at matching (os, model, group). Mutation: revert ActiveDeploymentForTarget to the global key → device A offered B → **FAIL**.

5.2 memory ↔ pgx contract parity (this session's HB-2/STORE-1 lesson)

The app-layer scope suite runs against **BOTH** backends via the shared contract suite (store/contract_test.go), so a memory-vs-pgx behavioral divergence is caught — the memory backend's isolation has no RLS safety net. Concurrency/race covers the re-scoped monotonicity + active-deployment keys and CreateAccountWithOwner atomicity (no orphan tenant).

5.3 The other mandatory suites

- **Super-admin** (27_* §4): bypass works only for super-admin; the RLS bypass is policy-based + **immediately revocable** (clear the GUC → same session re-scopes); audit written with the affected account_id; a forged is_super_admin claim/field is **ignored** (bootstrap-only). Each with a mutation that FAILs.
- **Migration/backfill** (27_* §5, pgx): COUNT(account_id IS NULL)=0 and COUNT(outside __default__)=0 after 003; composite-FK integrity; idempotent + crash-safe (SIGKILL mid-backfill → resumable).
- **UI host-render §11.4.170** (27_* §3): device-independent host-rendered pixels per screen×state×{light,dark}, dual-validated by golden image-diff **AND** OCR/vision oracle; value/token-equality tests FORBIDDEN as sole proof; the harnesses already exist [inherited, 23_* §4] — tenancy screens add fixtures; oracles self-validated golden-good/ golden-bad (§11.4.107(10)).
- **DDoS / stress+chaos / memory / benchmarking / HelixQA** per 27_* §1.

5.4 Two honest SKIPs by construction

The **RLS layer** is SKIP: `topology_unsupported` without a live Postgres, and the **byte-level device-apply e2e** is SKIP: `object_storage_absent` until M7's Storage seam — both tracked in the coverage ledger, never faked green. Isolation/scope/UI/super-admin/ migration are all fully provable **now** without either. Final gate: §11.4.185 **manual QA-team confirmation** after the autonomous suites are GREEN.

6. Phased implementation plan (T2 / feature/ multi-tenant-accounts)

SSOT: 30_* (8 milestones) = PRODUCTION_READINESS_PLAN P3 (M1–M8). Isolated §11.4.167 big-feature stream, own `.git` (§11.4.179), trunk merged in regularly (§11.4.188, force-push forbidden §11.4.113), merged to main only after operator approval + M8 §11.4.40 retest + independent review GO + §11.4.185 manual QA. Effort = T-shirt estimate only (§11.4.172 — no calendar precision until velocity is measured).

Serial spine M1→M2→M3, then M4/M5/M6 fan out in parallel; M7's storage seam is a cross-cut pulled EARLY (right after M3). Longest path: M1→M2→M3→M7-storage-seam→M5/M6 e2e→M8.

M	Milestone	Derives from	Effort	Depends on	Decision-gate input
M1	Data model + 002/003 migrations + store.Repository account scoping (both backends) + RLS + composite FK + CreateAccountWithOwner + the §2.5 <code>api_keys.project_id</code> fix	20_*	XL	— (root)	OQ-4, TEXT-vs-UUID
M2	AuthZ: <code>account_id</code> claim + <code>requireAccountAccess/requireSuperAdmin</code> + super-admin flag + policy-based RLS bypass + single role-vocab SSOT	21_*, 20_* §6	L	M1	OQ-2

M	Milestone	Derives from	Effort	Depends on	Decision-gate input
	+ SEC-1 fail-fast if not already on T1				
M3	Scope every OTA route (claim on hot path) + /admin/* API + /auth/accounts+/auth/select-account + GET /artifacts + NOT_FOUND cross-tenant	22_*	L	M1+M2	404-vs-403
M4	UI: sign-in split + account switcher (above project switcher) + super-admin console, both SPAs, host-render-proven light+dark; add super_admin to the dashboard Role union FIRST; fix the OTA-Manager sidebar.tsx "developer" literal	23_*, §11.4.162/.170	L	M2+M3	OpenCode-vs-OpenDesign
M5	Project-side integration CLI server/cmd/helix-ota (extend the upload path; key = exchange credential; §11.4.10 redaction + doctor)	24_*	M	M3 (+M7 for e2e)	CLI auth A/B
M6	Device/System update client + setup wizard (ota-android-agent): server-minted (account,project,device) token, notify/consent wizard, per-account signature verify	25_*, §11.4.170	XL	M1–M3 (+M7 for byte e2e)	device enrollment A/B
M7	Object-storage seam (MinIO dev via containers submodule; real StorageRef; per-account bucket/prefix; signed URLs) + per-account signing-key	26_*, 24_* §4	L	M1 (M5/M6 consume)	per-account keys

M	Milestone	Derives from	Effort	Depends on	Decision-gate input
	registry + credential hardening + SEC-1 (if deferred)				
	Full §11.4.40 retest from clean baseline + coverage				
M8	ledger + independent review GO + §11.4.185 manual-QA final gate + operator-approval merge	27_*, §11.4.40/.185	M-L	M1-M7	—

Role-vocabulary reconciliation (§11.4.186, danger zone 30_* §3.2)

rides M2: the same role set lives in five places and diverges — server token.go omits super_admin; dashboard Role union omits super_admin (the console cannot be gated there until fixed); the OTA-Manager sidebar.tsx gates on a "developer" literal not in the union. M2 makes the server token.go set the single SSOT and reconciles both SPAs in one sweep before any super-admin UI (M4) lands.

Tracked pre-scoping audit (30_* §3.3): the rollout subsystem's tenancy is UNCONFIRMED (internal/rollout/) — audit rollout/store.go before scoping its routes; on the evidence read it takes deployment ids, not account ids, but this is an unscoped path the plan cannot certify without the audit.

7. Open decisions for the operator (K12) — resolve BEFORE M1

Consolidated from 28_* §2 + 30_* §5. Each carries the design-set recommendation + tradeoff; **none is silently decided** (§11.4.66). The two **RESIDUAL-BLOCKING** reconciliations (#H, #J) this proposal already resolves in §2.5/§3.3 — confirm them.

#	Decision	Recommended (with rationale)	Tradeoff to accept	B
OQ-2	Permission-model shape (RBAC / ABAC / hybrid)	RBAC-first role+scope hybrid — tenant-isolation predicate → RBAC matrix → thin attribute-deny; reuses the shipped role hierarchy +	fixed 3-role enum caps flexibility at first ship; full ABAC (Cedar/	M

#	Decision	Recommended (with rationale)	Tradeoff to accept	B
		RESOURCE_PERMISSIONS grid verbatim; closes the cross-account hole by construction; grows to custom roles via reserved tables (no 2nd data-model change)	OPA) is a later escalation	
OQ-3	Identity source	Local accounts only (super-admin-provisioned; no self-registration — the mandate implies it, <code>User.password_hash</code> models it)	federation (OIDC/SAML) is a future additive <code>User.external_idp</code> column, not built now	M
OQ-4	Migrate existing data under accounts	Default-account backfill (big-bang → NOT NULL) — removes the nullable-isolation-hole window; near-free (store is in-memory-by-default, barely populated)	one coordinated 003 cutover; flip to additive-nullable ONLY if a target holds real prod data AND zero-downtime is mandatory	M
Per-account signing keys	Global key vs per-account registry	Per-account ed25519 verify-key registry, global key as migration fallback — <code>resolvePublicKey(accountID)</code> from server config/registry; trust boundary unchanged (key never from request; only the <i>lookup key</i> gains an account dimension)	a registry + per-account key lifecycle; a shared global key lets any account's signer sign for the platform (unacceptable long-term)	M
Id type	TEXT vs UUID	TEXT to mirror the shipped store (avoids a store-wide id-type migration; ids appointed as today); 20_* §4 states the TEXT→UUID mapping for the canonical schema	forgoes server-side <code>gen_random_uuid()</code>	M
#H (resolve)		Add <code>api_keys.project_id</code> TEXT nullable (NULL = any	one additive column	M

#	Decision	Recommended (with rationale)	Tradeoff to accept	B
	api_keys.project_id absent from the SSOT DDL	project in the account) — completes the SSOT so the CLI's project-narrowed key model has a column		
#J (resolve)	Legacy no-claim token handling	Fail-closed on scoped routes (super-admin exempt); strike the token → __default__ fallback; keep the row → __default__ data backfill	live legacy sessions re-select an account once (self-clears within one 15-min TTL)	M

Secondary (recommendation noted, not blocking a whole milestone): sign-in split A (post-auth routing) vs B (separate super-admin door); CLI auth direct-key (A) vs token-exchange (B, recommended); device enrollment operator-minted (A, now) vs bootstrap → rotate (B, at scale); cross-tenant denial NOT_FOUND (recommended) vs 403; atomic-bootstrap CreateAccountWithOwner (A, now) vs generic WithTx (B). **OpenCode-vs-OpenDesign** is a naming clarification (12_* resolved: OpenCode = the coding *agent* used to build UI; OpenDesign = the mandated design-*token* system the UI consumes, §11.4.162) — confirm, does not block M4.

Honest boundary (§11.4.6)

- **This is a design + plan PROPOSAL, not an implementation, and not a completion claim.** No entity, column, migration, claim, route, middleware, UI, CLI, device client, test, or RLS policy described here exists in code yet. The whole feature is a to-be proposal across 20_*–27_*; no work starts until the operator approves this plan AND resolves the §7 gate (§11.4.66).
- **Not a second source of truth (§11.4.186).** This doc consolidates + sequences; the research/design set are the SSOTs and win on any shape conflict. 28_* already ran the §11.4.186 cross-doc consistency pass (verdict: CONDITIONAL PASS after #H + #J land, which §2.5/§3.3 adopt).
- **As-is grounding scope.** The file:line facts marked verified this session were re-read from non-test source: config.go (secret fallback 180-184; DB selection 112-116; TrustTLSProxy 88-110), token.go (14-20 roles, 31-36 claims, HMAC signer), middleware.go (62-97 auth/requireRole), users.go (8-48 static directory), store.go

(257-311 filters/Project/ProjectAccess, 315-429 Repository interface), handlers_project.go (37-92 requireProjectAccess/rank). Facts marked **[inherited]** (resolvePublicKey/handlers_artifact.go, audit_wire.go, StorageRef placeholder, the visual harnesses, internal/rollout/ tenancy) are cited to the research set and were **not** re-opened here; the rollout subsystem's tenancy remains **UNCONFIRMED** and must be audited before its routes are scoped.

- **Two layers are honestly non-green by construction today:** RLS needs a live Postgres (memory-backend isolation rests on L1+L2 only); byte-level device-apply e2e needs real object storage (absent — bytes validated then discarded). Both are surfaced, never green-over-a-gap.
- **SEC-1 status (verified):** config.go:180-184 **still silently falls back** to the known dev secret as read this session; the fail-fast is in-flight on T1/main, and the account claim (M2) hard-depends on it landing first — a predictable secret defeats tenant isolation entirely.
- **Estimates are estimates (§11.4.172).** §6 T-shirt sizes are not calendar commitments; replace with measured-velocity ranges after M1–M2.

Provenance / sources

Synthesized from the operator-verified research set (each carries its own ## Sources verified 2026-07-10 footer — AWS RLS, OWASP BOLA/ Multi-Tenant cheat sheet, Microsoft Azure Architecture Center, WorkOS, Auth0, AWS Well-Architected SaaS Lens, NIST/TUF, Playwright): docs/research/accounts/{00_INDEX,10,11,12,20,21,22,23,24,25,26,27,28,30}.md, plus docs/planning/PRODUCTION_READINESS_PLAN.md §2.1/§5/K12. No new external research was performed for this consolidation (it is an internal synthesis, not a §11.4.99 latest-source pass); current-state facts are first-hand re-reads of the cited server source.